ROCODEFEST · BACKEND DATA MODEL

OLTP Schema — reconciled with the R.O.O.T. tree export

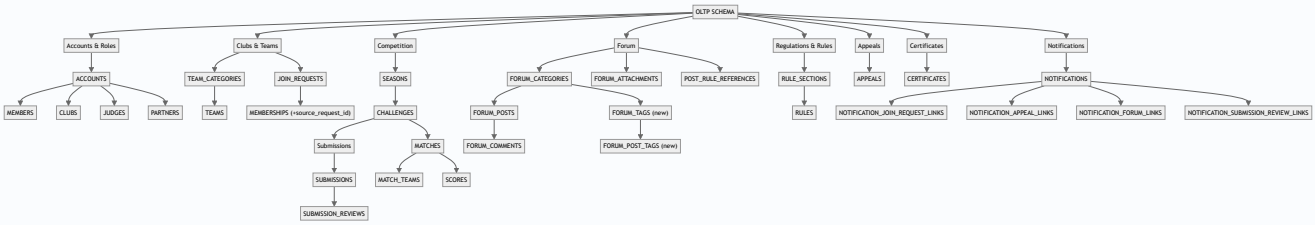
This revision of `01_oltp_schema.sql` was checked node by node and link by link against the tree exported from R.O.O.T. (journal "Backend (Festivalul RoCodeFest)", root node `OLTP SCHEMA`, exported 2026-08-25). Of the 39 entities and 50 relationships in that export, all but three were already implemented exactly as modelled. The three differences are below.

WHAT CHANGED

1. **Forum tags** — the tree's `FORUM TAG` node (`tag_id`, `post_id`, `category_id` in one node) is normalized into `forum_tags` (a category-scoped tag lookup) plus `forum_post_tags` (the post↔tag junction). `category_id` is deliberately not repeated on the junction — it already lives on both `forum_posts` and `forum_tags`, and a third copy could drift.
2. **Membership provenance** — the tree parents `MEMBERSHIPS` under `JOIN REQUESTS` ("apartenența se creează dintr-un JOIN REQUEST aprobat"), a link the SQL did not carry. Added `memberships.source_request_id → join_requests(request_id)`, nullable (a club may add a member directly, with no request), `ON DELETE SET NULL`, with a partial unique index so one approved request can never be replayed into several roster rows. `join_requests` also moved up into the Clubs & Teams section — both because the tree groups it there and because it must now be declared before `memberships`.
3. **Seasons → Matches, deliberately not added** — the tree carries this edge, but `matches` holds only `challenge_id`; season is reached transitively through `challenges`. Storing `season_id` on `matches` too would be a denormalization that can contradict its own challenge. This is likely why the tree flags the edge as needing repair.

Whole schema, as a tree

The export's hierarchy, reproduced exactly — containment only, no FK arrows, so no lines cross. New and changed pieces are marked.



Full DDL — 01_oltp_schema.sql

Complete, current source of truth.

```
-- ============================================================================
-- ROcodeFest — OLTP Schema (PostgreSQL, 3NF)
-- ============================================================================
-- Operational schema backing the Node.js API: accounts/roles, clubs &
-- teams, join requests, forum, regulations, submissions (with
-- judge-assigned reviews and resubmission versioning), competition
-- (challenges/matches/scores), appeals, certificates, notifications.
--
-- Conventions used throughout:
--   * Surrogate keys use `GENERATED ALWAYS AS IDENTITY` (modern
--     Postgres, replaces legacy SERIAL) except where a natural key
--     already makes a good primary key (subtype tables keyed by
--     account_id).
--   * Every FK has an explicit index unless it's already covered by a
--     PRIMARY KEY or UNIQUE constraint (Postgres does NOT auto-index
--     FK columns, only PK/UNIQUE ones).
--   * Small closed vocabularies that don't need extra admin-editable
--     attributes are Postgres ENUM types; vocabularies that carry
--     extra attributes (e.g. a category's max member count) are
--     lookup TABLES instead, so they can be edited without a
--     migration. Both choices are noted inline where they happen.
--   * Business rules that require aggregating across rows (team
--     capacity by category, max 3 invalid appeals per team) can't be
--     expressed as a plain CHECK constraint in Postgres, so they're
--     enforced with small BEFORE INSERT triggers instead. This is
--     called out at each trigger.
-- ============================================================================

CREATE EXTENSION IF NOT EXISTS citext; -- case-insensitive email uniqueness

-- ----------------------------------------------------------------------------
-- ENUM types
-- ----------------------------------------------------------------------------

CREATE TYPE account_role AS ENUM ('member', 'club', 'judge', 'partner');
-- 'member'  = a Competitor -- the only role that can join a team.
-- 'partner' covers BOTH the "Partner" and "Speaker" registration paths
--           (see partners.partner_type below). Both are external
--           collaborators with the same core shape (org/contact +
```

```
--            a message), so one subtype table with a type
--            discriminator avoids a near-duplicate `speakers` table.
--            If speaker-specific structured fields grow later (talk
--            topic, availability slots), split it out into its own
--            subtype table at that point.

CREATE TYPE partner_type AS ENUM ('sponsor', 'speaker', 'mentor', 'vendor', 'other');

-- Shared by join_requests AND appeals -- both are literally the same
-- pending/accepted/rejected decision workflow, so one ENUM is reused
-- instead of declaring two near-identical types.
CREATE TYPE request_status AS ENUM ('pending', 'accepted', 'rejected');

CREATE TYPE challenge_stage AS ENUM ('city_challenge', 'final');
CREATE TYPE match_status AS ENUM ('scheduled', 'in_progress', 'completed', 'cancelled');
CREATE TYPE certificate_type AS ENUM ('participation', 'winner', 'runner_up', 'special_award');
CREATE TYPE notification_type AS ENUM (
  'join_request_received', 'join_request_accepted', 'join_request_rejected',
  'appeal_accepted', 'appeal_rejected',
  'announcement_posted',
  'comment_posted', 'reply_posted',
  'submission_assigned', 'judge_feedback_given'
);
-- submission_assigned / judge_feedback_given are both lifecycle events on
-- one submission_reviews row (see Submissions section) -- that table is
-- both "which judge evaluates this submission" and "what feedback did
-- they leave", so both notification types point at the same source row.

-- ======================================================================
-- Accounts & roles
-- ======================================================================
-- Table-per-subtype (class-table inheritance): `accounts` holds
-- everything every login needs (credentials, role, audit columns);
-- one subtype table per role holds the fields specific to that role.
-- account_id is BOTH the subtype table's primary key AND its foreign
-- key back to accounts, so a `members` row IS a 1:1 extension of an
-- `accounts` row -- there is exactly one place email/password live
-- (3NF: every non-key column depends on the whole key and nothing but
-- the key).
--
-- A role is chosen once at registration and stored on accounts.role
-- (single column, not a many-to-many junction) because that matches
-- how registration actually works today: you sign up AS a club, OR a
-- competitor, OR a judge, OR a partner/speaker -- never several at
-- once. If multi-role accounts are ever needed, swap accounts.role for
-- an account_roles(account_id, role) junction without touching the
-- four subtype tables below.

CREATE TABLE accounts (
  account_id     BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email          CITEXT NOT NULL,
  password_hash  TEXT NOT NULL,
  role           account_role NOT NULL,
  is_active      BOOLEAN NOT NULL DEFAULT TRUE,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
  CONSTRAINT uq_accounts_email UNIQUE (email)
);

CREATE TABLE members (
  account_id  BIGINT PRIMARY KEY REFERENCES accounts(account_id) ON DELETE CASCADE,
  full_name   VARCHAR(150) NOT NULL,
  city        VARCHAR(120),
  bio         TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE clubs (
  account_id       BIGINT PRIMARY KEY REFERENCES accounts(account_id) ON DELETE CASCADE,
  club_name        VARCHAR(150) NOT NULL,
  city             VARCHAR(120),
  coach_name       VARCHAR(150),
  -- Own IDENTITY sequence (independent of account_id): a stable,
  -- always-unique "founding number" assigned in registration order.
  -- Doing this in the DB (not in application code) is what actually
  -- fixes the duplicate "#1" badge bug we hit in the frontend demo,
  -- where the number was recomputed client-side as `clubs.length + 1`
  -- and collided whenever local demo data was reset.
  founding_number  INT GENERATED ALWAYS AS IDENTITY,
  notes            TEXT,
  created_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_clubs_name UNIQUE (club_name),
  CONSTRAINT uq_clubs_founding_number UNIQUE (founding_number)
);

CREATE TABLE judges (
  account_id  BIGINT PRIMARY KEY REFERENCES accounts(account_id) ON DELETE CASCADE,
  full_name   VARCHAR(150) NOT NULL,
  expertise   VARCHAR(200),
  bio         TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE partners (
  account_id    BIGINT PRIMARY KEY REFERENCES accounts(account_id) ON DELETE CASCADE,
  org_name      VARCHAR(150),
  contact_name  VARCHAR(150) NOT NULL,
  partner_type  partner_type NOT NULL, -- distinguishes Partner vs Speaker (and future types)
  message       TEXT,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ========================================================================
-- Clubs & teams
-- ========================================================================

-- Lookup table, not an ENUM: each category carries an extra business
-- attribute (max_members) that legitimately varies and may need
-- editing (e.g. bumping Programming's cap) without a schema migration.
CREATE TABLE team_categories (
  category_id  SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  code         VARCHAR(20) NOT NULL,
```

```
  label        VARCHAR(60) NOT NULL,
  max_members  SMALLINT NOT NULL CHECK (max_members > 0),
  CONSTRAINT uq_team_categories_code UNIQUE (code)
);

-- Robotics/Programming caps per spec; Design/Other caps are a
-- reasonable default (assumption) -- change via UPDATE, no migration.
INSERT INTO team_categories (code, label, max_members) VALUES
  ('ROBOTICS',    'Robotics',    5),
  ('PROGRAMMING', 'Programming', 3),
  ('DESIGN',      'Design',      5),
  ('OTHER',       'Other',       5);

CREATE TABLE teams (
  team_id             BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  club_id             BIGINT NOT NULL REFERENCES clubs(account_id) ON DELETE CASCADE,
  category_id         SMALLINT NOT NULL REFERENCES team_categories(category_id),
  team_name           VARCHAR(150) NOT NULL,
  participation_code  VARCHAR(20) NOT NULL, -- public-facing team code (e.g. shown on badges/certificates)
  is_active           BOOLEAN NOT NULL DEFAULT TRUE,
  created_at          TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_teams_participation_code UNIQUE (participation_code),
  CONSTRAINT uq_teams_club_name UNIQUE (club_id, team_name) -- a club can't register two teams with the same
name
);
CREATE INDEX ix_teams_club_id ON teams(club_id);
CREATE INDEX ix_teams_category_id ON teams(category_id);

-- Join requests belong to this domain (a request is addressed to a
-- team), and they're declared BEFORE memberships because an approved
-- request is what produces a membership -- see
-- memberships.source_request_id below.
CREATE TABLE join_requests (
  request_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  member_id     BIGINT NOT NULL REFERENCES members(account_id) ON DELETE CASCADE,
  team_id       BIGINT NOT NULL REFERENCES teams(team_id) ON DELETE CASCADE,
  status        request_status NOT NULL DEFAULT 'pending',
  message       TEXT,
  requested_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  decided_at    TIMESTAMPTZ,
  decided_by    BIGINT REFERENCES accounts(account_id) -- the club account that accepted/rejected
);
CREATE INDEX ix_join_requests_member_id ON join_requests(member_id);
CREATE INDEX ix_join_requests_team_id ON join_requests(team_id);
CREATE INDEX ix_join_requests_decided_by ON join_requests(decided_by);
-- Only one PENDING request per (member, team); a member CAN re-request
-- after a rejection, so this must be a partial unique index scoped to
-- status='pending', not a full unique constraint on (member_id, team_id).
CREATE UNIQUE INDEX uq_join_requests_pending
  ON join_requests(member_id, team_id) WHERE status = 'pending';

-- Many-to-many members <-> teams. A surrogate PK (not a composite
-- (team_id, member_id) PK) is used because a membership carries its
-- own attributes (role_title, joined_at/left_at) and needs to support
-- history: a member can leave and later rejoin the same team, which
-- would violate a plain composite-PK/UNIQUE if we tried to reuse the
```

```
-- same row identity for both stints.
CREATE TABLE memberships (
  membership_id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  team_id        BIGINT NOT NULL REFERENCES teams(team_id) ON DELETE CASCADE,
  member_id      BIGINT NOT NULL REFERENCES members(account_id) ON DELETE CASCADE,
  -- Provenance: which approved join_request created this membership.
  -- NULLABLE on purpose -- a club can also add a member to one of its
  -- own teams directly, with no request in between, and historical
  -- rows predating this column have no request to point at. It is NOT
  -- a duplicate of (team_id, member_id): a member who left and
  -- rejoined has two membership rows, each traceable to the specific
  -- request that produced it. ON DELETE SET NULL so purging old
  -- request history never deletes an active roster row.
  source_request_id  BIGINT REFERENCES join_requests(request_id) ON DELETE SET NULL,
  role_title     VARCHAR(60), -- e.g. 'Captain'; free text, optional
  joined_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  left_at        TIMESTAMPTZ -- NULL while the membership is active
);
CREATE INDEX ix_memberships_team_id ON memberships(team_id);
CREATE INDEX ix_memberships_member_id ON memberships(member_id);
CREATE INDEX ix_memberships_source_request_id ON memberships(source_request_id);
-- Only one ACTIVE membership per (team, member) at a time; a partial
-- unique index (not a table constraint) so leaving and rejoining is
-- allowed -- a full UNIQUE(team_id, member_id) would permanently block
-- rejoining after a single departure.
CREATE UNIQUE INDEX uq_memberships_active_team_member
  ON memberships(team_id, member_id) WHERE left_at IS NULL;
-- One request can only ever produce ONE membership -- a partial unique
-- index (skipping the NULLs, which a plain UNIQUE would treat as all
-- distinct anyway) so an approved request can't be replayed into
-- several roster rows.
CREATE UNIQUE INDEX uq_memberships_source_request
  ON memberships(source_request_id) WHERE source_request_id IS NOT NULL;

-- ======================================================================
-- Forum
-- ======================================================================

-- Lookup table, not an ENUM: forum categories are the kind of thing an
-- organizer wants to add/rename/reorder from an admin screen without a
-- deploy.
CREATE TABLE forum_categories (
  category_id  SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  code         VARCHAR(30) NOT NULL,
  label        VARCHAR(60) NOT NULL,
  sort_order   SMALLINT NOT NULL DEFAULT 0,
  CONSTRAINT uq_forum_categories_code UNIQUE (code)
);

CREATE TABLE forum_posts (
  post_id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  category_id        SMALLINT NOT NULL REFERENCES forum_categories(category_id),
  author_account_id  BIGINT NOT NULL REFERENCES accounts(account_id),
  title              VARCHAR(200) NOT NULL,
  body               TEXT,
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
  edited_at          TIMESTAMPTZ,
  is_deleted         BOOLEAN NOT NULL DEFAULT FALSE -- soft delete: preserves thread/reply integrity for
moderation
);
CREATE INDEX ix_forum_posts_category_id ON forum_posts(category_id);
CREATE INDEX ix_forum_posts_author_id ON forum_posts(author_account_id);
-- NOTE: post "type" (member vs club) is intentionally NOT a stored
-- column here. It's derived by joining author_account_id -> accounts.
-- role, so it can never drift out of sync with the account's actual
-- role -- storing a copy would be redundant, derivable data (violates
-- 3NF: it wouldn't depend on this table's own key).

CREATE TABLE forum_comments (
  comment_id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  post_id            BIGINT NOT NULL REFERENCES forum_posts(post_id) ON DELETE CASCADE,
  parent_comment_id  BIGINT REFERENCES forum_comments(comment_id) ON DELETE CASCADE, -- optional nested replies
  author_account_id  BIGINT NOT NULL REFERENCES accounts(account_id),
  body               TEXT NOT NULL,
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  edited_at          TIMESTAMPTZ,
  is_deleted         BOOLEAN NOT NULL DEFAULT FALSE
);
CREATE INDEX ix_forum_comments_post_id ON forum_comments(post_id);
CREATE INDEX ix_forum_comments_parent_id ON forum_comments(parent_comment_id);
CREATE INDEX ix_forum_comments_author_id ON forum_comments(author_account_id);

-- Metadata only, per spec -- actual file bytes live in object storage
-- (S3-compatible bucket etc.), referenced elsewhere by the app; this
-- table never stores file content. Can attach to a post OR a comment,
-- never neither/both: modelled as two nullable FKs plus a CHECK using
-- num_nonnulls() rather than a polymorphic type+id pair, so both
-- relationships stay real, indexable foreign keys.
CREATE TABLE forum_attachments (
  attachment_id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  post_id        BIGINT REFERENCES forum_posts(post_id) ON DELETE CASCADE,
  comment_id     BIGINT REFERENCES forum_comments(comment_id) ON DELETE CASCADE,
  file_name      VARCHAR(255) NOT NULL,
  mime_type      VARCHAR(100) NOT NULL,
  size_bytes     INTEGER NOT NULL CHECK (size_bytes > 0 AND size_bytes <= 1048576), -- <= 1 MB, enforced in-DB
  uploaded_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT ck_forum_attachments_one_parent CHECK (num_nonnulls(post_id, comment_id) = 1)
);
CREATE INDEX ix_forum_attachments_post_id ON forum_attachments(post_id);
CREATE INDEX ix_forum_attachments_comment_id ON forum_attachments(comment_id);

-- Lookup table, scoped to a category (not global) -- a "Robotics" tag
-- and a "Forum" tag can both read as e.g. "Help Wanted" without
-- colliding, and an admin can rename/retire tags per category without
-- touching unrelated ones.
CREATE TABLE forum_tags (
  tag_id       SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  category_id  SMALLINT NOT NULL REFERENCES forum_categories(category_id),
  label        VARCHAR(40) NOT NULL,
  CONSTRAINT uq_forum_tags_category_label UNIQUE (category_id, label)
);
CREATE INDEX ix_forum_tags_category_id ON forum_tags(category_id);
```

```
-- Many-to-many post <-> tag. category_id is deliberately NOT repeated
-- here: a post's category already lives on forum_posts.category_id,
-- and a tag's category lives on forum_tags.category_id above --
-- copying it onto every (post, tag) row would let a post's category
-- and its tags' category drift apart with no single source of truth.
-- (Enforcing "a post can only use tags from its own category" is an
-- application-layer/CHECK-trigger concern, not a column.)
CREATE TABLE forum_post_tags (
  post_id  BIGINT NOT NULL REFERENCES forum_posts(post_id) ON DELETE CASCADE,
  tag_id   SMALLINT NOT NULL REFERENCES forum_tags(tag_id) ON DELETE CASCADE,
  PRIMARY KEY (post_id, tag_id)
);
CREATE INDEX ix_forum_post_tags_tag_id ON forum_post_tags(tag_id);

-- ======================================================================
-- Regulations & rules
-- ======================================================================

CREATE TABLE rule_sections (
  section_id  SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  code        VARCHAR(5) NOT NULL, -- e.g. 'G' (General), 'S' (Season/Scoring), 'A' (Appeals), 'E' (Eligibility)
  title       VARCHAR(100) NOT NULL,
  sort_order  SMALLINT NOT NULL DEFAULT 0,
  CONSTRAINT uq_rule_sections_code UNIQUE (code)
);

CREATE TABLE rules (
  rule_id        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  section_id     SMALLINT NOT NULL REFERENCES rule_sections(section_id),
  code           VARCHAR(10) NOT NULL, -- e.g. 'G4', 'A21', 'E2' -- the citable business key everyone actually
uses
  title          VARCHAR(200) NOT NULL,
  body           TEXT NOT NULL,
  version_label  VARCHAR(30),
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_rules_code UNIQUE (code)
);
CREATE INDEX ix_rules_section_id ON rules(section_id);
-- `code` duplicates the section's prefix (G4 starts with the 'G'
-- section code) -- a small, deliberate denormalization. Composing it
-- from section_id + a sequence number on every read would be more
-- "pure" 3NF, but codes are how staff and users actually reference
-- rules (forum posts, appeals), so storing the authoritative display
-- code directly avoids fragile string-building logic scattered across
-- the app.

-- Many-to-many: a forum post can cite several rules, and a rule can be
-- cited by many posts. Composite PK (no surrogate key needed) because
-- the pair itself is the whole fact being recorded.
CREATE TABLE post_rule_references (
  post_id     BIGINT NOT NULL REFERENCES forum_posts(post_id) ON DELETE CASCADE,
  rule_id     BIGINT NOT NULL REFERENCES rules(rule_id) ON DELETE CASCADE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (post_id, rule_id)
);
```

```
CREATE INDEX ix_post_rule_references_rule_id ON post_rule_references(rule_id);

-- ======================================================================
-- Competition: seasons, challenges, matches, scores
-- ======================================================================

CREATE TABLE seasons (
  season_id   SMALLINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name        VARCHAR(60) NOT NULL,
  start_date  DATE NOT NULL,
  end_date    DATE,
  is_active   BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_seasons_name UNIQUE (name)
);

CREATE TABLE challenges (
  challenge_id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  season_id            SMALLINT NOT NULL REFERENCES seasons(season_id),
  category_id          SMALLINT REFERENCES team_categories(category_id), -- NULL = applies to every category
  name                 VARCHAR(150) NOT NULL,
  description          TEXT,
  stage                challenge_stage NOT NULL,
  submission_deadline  TIMESTAMPTZ
);
CREATE INDEX ix_challenges_season_id ON challenges(season_id);
CREATE INDEX ix_challenges_category_id ON challenges(category_id);

CREATE TABLE matches (
  match_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  challenge_id  BIGINT NOT NULL REFERENCES challenges(challenge_id) ON DELETE CASCADE,
  round_name    VARCHAR(100),
  scheduled_at  TIMESTAMPTZ,
  location      VARCHAR(150),
  status        match_status NOT NULL DEFAULT 'scheduled'
);
CREATE INDEX ix_matches_challenge_id ON matches(challenge_id);

-- Many-to-many matches <-> teams (a match can involve several teams:
-- head-to-head heats, multi-team rounds, etc.).
CREATE TABLE match_teams (
  match_id  BIGINT NOT NULL REFERENCES matches(match_id) ON DELETE CASCADE,
  team_id   BIGINT NOT NULL REFERENCES teams(team_id) ON DELETE CASCADE,
  PRIMARY KEY (match_id, team_id)
);
CREATE INDEX ix_match_teams_team_id ON match_teams(team_id);

CREATE TABLE scores (
  score_id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  match_id            BIGINT NOT NULL,
  team_id             BIGINT NOT NULL,
  judge_id            BIGINT NOT NULL REFERENCES judges(account_id),
  innovation_score    SMALLINT NOT NULL CHECK (innovation_score BETWEEN 1 AND 10),
  execution_score     SMALLINT NOT NULL CHECK (execution_score BETWEEN 1 AND 10),
  presentation_score  SMALLINT NOT NULL CHECK (presentation_score BETWEEN 1 AND 10),
  teamwork_score      SMALLINT NOT NULL CHECK (teamwork_score BETWEEN 1 AND 10),
  -- Generated column: total is always innovation+execution+presentation
```

```
  -- +teamwork, so it's computed and stored by Postgres itself rather
  -- than trusted to app code -- it can never drift from its inputs.
  total_score         SMALLINT GENERATED ALWAYS AS
                         (innovation_score + execution_score + presentation_score + teamwork_score) STORED,
  comment             TEXT,
  scored_at           TIMESTAMPTZ NOT NULL DEFAULT now(),
  -- Composite FK: a score can only be recorded for a (match, team) pair
  -- that is actually registered in match_teams -- prevents scoring a
  -- team that isn't in that match.
  CONSTRAINT fk_scores_match_team FOREIGN KEY (match_id, team_id)
    REFERENCES match_teams(match_id, team_id) ON DELETE CASCADE,
  -- One score row per judge per team per match; a judge revising their
  -- score should UPDATE this row, not insert a second one.
  CONSTRAINT uq_scores_match_team_judge UNIQUE (match_id, team_id, judge_id)
);
CREATE INDEX ix_scores_judge_id ON scores(judge_id);
CREATE INDEX ix_scores_team_id ON scores(team_id);
CREATE INDEX ix_scores_match_id ON scores(match_id);

-- ======================================================================
-- Appeals
-- ======================================================================

CREATE TABLE appeals (
  appeal_id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  team_id                BIGINT NOT NULL REFERENCES teams(team_id) ON DELETE CASCADE,
  match_id                BIGINT REFERENCES matches(match_id),
  score_id                BIGINT REFERENCES scores(score_id), -- the specific score being appealed, if any
  rule_id                 BIGINT REFERENCES rules(rule_id),   -- optional cited rule (e.g. G4, A21)
  filed_by_account_id     BIGINT NOT NULL REFERENCES accounts(account_id),
  livestream_timestamp    INTERVAL, -- elapsed time into the stream, e.g. '01:42:30'
  reason                  TEXT NOT NULL,
  status                  request_status NOT NULL DEFAULT 'pending',
  decided_at               TIMESTAMPTZ,
  decided_by_account_id    BIGINT REFERENCES accounts(account_id),
  created_at               TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_appeals_team_id ON appeals(team_id);
CREATE INDEX ix_appeals_match_id ON appeals(match_id);
CREATE INDEX ix_appeals_score_id ON appeals(score_id);
CREATE INDEX ix_appeals_rule_id ON appeals(rule_id);
CREATE INDEX ix_appeals_filed_by ON appeals(filed_by_account_id);
CREATE INDEX ix_appeals_decided_by ON appeals(decided_by_account_id);

-- ======================================================================
-- Submissions  (part of the Competition domain: a submission is always
--               "for" a challenge -- kept as its own section here only
--               because it must be declared after appeals/scores)
-- ======================================================================

-- Versioned, not edit-in-place: a judge can send a submission back for
-- rework (see submission_reviews.requires_resubmission below), and the
-- team's next attempt for the same challenge needs its own row so the
-- review history isn't overwritten. `version` is assigned by
-- trg_submissions_version (see Triggers) rather than trusted to the
-- client, so it can't be skipped or duplicated by a racing request.
```

```
CREATE TABLE submissions (
  submission_id            BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  team_id                  BIGINT NOT NULL REFERENCES teams(team_id) ON DELETE CASCADE,
  challenge_id             BIGINT NOT NULL REFERENCES challenges(challenge_id) ON DELETE CASCADE,
  version                  SMALLINT NOT NULL DEFAULT 1,
  title                    VARCHAR(200) NOT NULL,
  description              TEXT,
  link_url                 TEXT,
  submitted_by_account_id  BIGINT NOT NULL REFERENCES accounts(account_id),
  created_at               TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at               TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_submissions_team_challenge_version UNIQUE (team_id, challenge_id, version)
);
CREATE INDEX ix_submissions_challenge_id ON submissions(challenge_id);
CREATE INDEX ix_submissions_submitted_by ON submissions(submitted_by_account_id);
-- (team_id already indexed via the UNIQUE constraint above.)

-- Connects a judge to a specific submission: the row's existence is the
-- assignment ("this judge evaluates this submission"), and `feedback`
-- filled in later is the review. One judge can review the same team's
-- submission again for a resubmission -- that's just a new submissions
-- row (new version) getting its own submission_reviews row, so a
-- judge's full history for a team/challenge is every review row across
-- every version, joinable straight back through submission_id.
CREATE TABLE submission_reviews (
  review_id              BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  submission_id          BIGINT NOT NULL REFERENCES submissions(submission_id) ON DELETE CASCADE,
  judge_id               BIGINT NOT NULL REFERENCES judges(account_id) ON DELETE CASCADE,
  feedback                TEXT, -- NULL = assigned, not yet reviewed
  requires_resubmission   BOOLEAN NOT NULL DEFAULT FALSE,
  reviewed_at             TIMESTAMPTZ, -- NULL until feedback is actually given
  assigned_at             TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_submission_reviews_submission_judge UNIQUE (submission_id, judge_id)
);
CREATE INDEX ix_submission_reviews_submission_id ON submission_reviews(submission_id);
CREATE INDEX ix_submission_reviews_judge_id ON submission_reviews(judge_id);

-- ====================================================================
-- Certificates
-- ====================================================================

CREATE TABLE certificates (
  certificate_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  team_id           BIGINT NOT NULL REFERENCES teams(team_id) ON DELETE CASCADE,
  member_id         BIGINT REFERENCES members(account_id) ON DELETE CASCADE, -- NULL = team-level certificate
  season_id         SMALLINT NOT NULL REFERENCES seasons(season_id),
  certificate_type  certificate_type NOT NULL DEFAULT 'participation',
  serial_code       VARCHAR(40) NOT NULL, -- verifiable public ID, e.g. 'RCFEST-2026-000123'
  file_url          TEXT,                 -- generated PDF/PNG in object storage
  issued_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_certificates_serial UNIQUE (serial_code)
);
CREATE INDEX ix_certificates_team_id ON certificates(team_id);
CREATE INDEX ix_certificates_member_id ON certificates(member_id);
CREATE INDEX ix_certificates_season_id ON certificates(season_id);
-- "Issued per team OR per participant" is modelled with one table and
```

```
-- a nullable member_id rather than two near-identical tables. Because
-- a plain UNIQUE constraint treats every NULL as distinct (so it would
-- NOT stop two team-level certs of the same type existing), uniqueness
-- is split into two partial indexes instead:
CREATE UNIQUE INDEX uq_certificates_team_level
  ON certificates(team_id, season_id, certificate_type) WHERE member_id IS NULL;
CREATE UNIQUE INDEX uq_certificates_participant_level
  ON certificates(team_id, member_id, season_id, certificate_type) WHERE member_id IS NOT NULL;

-- ======================================================================
-- Notifications
-- ======================================================================
-- Placed last on purpose: the event-link tables below cite almost every
-- other domain (join requests, appeals, the forum, submission reviews),
-- so they have to come after all of them exist. `notifications` itself
-- has no such dependency and could live anywhere, but stays next to its
-- links for readability.
--
-- `notifications` is the generic destination, mirroring the
-- accounts/subtype-table pattern used at the top of this file: an
-- inbox row for one account -- who it's for, its text, its read state,
-- when it's allowed to be delivered -- with no notion of *why* it
-- exists or what it's about.
CREATE TABLE notifications (
  notification_id       BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  recipient_account_id  BIGINT NOT NULL REFERENCES accounts(account_id) ON DELETE CASCADE,
  title                 VARCHAR(150) NOT NULL,
  body                  TEXT,
  -- Delivery buffer: NULL/now() = deliver immediately; set in the
  -- future to hold delivery until then. Used for comment/reply
  -- notifications specifically -- the app inserts with
  -- `notify_after = now() + interval '30 minutes'` so a flurry of
  -- edits or follow-up replies in that window collapses into one
  -- delivered notification instead of spamming the recipient. A
  -- dispatch worker only sends rows where notify_after <= now().
  notify_after           TIMESTAMPTZ NOT NULL DEFAULT now(),
  is_read                BOOLEAN NOT NULL DEFAULT FALSE,
  created_at             TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX ix_notifications_recipient ON notifications(recipient_account_id, is_read);
CREATE INDEX ix_notifications_notify_after ON notifications(notify_after) WHERE is_read = FALSE;

-- What each notification is *about* lives in one of four separate,
-- domain-scoped link tables rather than one shared table with a
-- nullable FK per source. Forum activity and appeals (for example)
-- have nothing to do with each other, so they don't share a table --
-- each link table only knows about its own domain's source rows. Every
-- link table is a 1:1 extension of `notifications` (same shape as
-- members/clubs/judges/partners extending accounts): notification_id
-- is both its PK and its FK back to notifications. A shared trigger
-- (enforce_single_notification_link, see Triggers) checks across all
-- four tables that a given notification_id has exactly one link total,
-- since Postgres can't express a cross-table uniqueness constraint
-- declaratively.
CREATE TABLE notification_join_request_links (
  notification_id  BIGINT PRIMARY KEY REFERENCES notifications(notification_id) ON DELETE CASCADE,
```

```
  event_type       notification_type NOT NULL, -- join_request_received/accepted/rejected
  join_request_id  BIGINT NOT NULL REFERENCES join_requests(request_id) ON DELETE CASCADE
);
CREATE INDEX ix_notification_join_request_links_request ON notification_join_request_links(join_request_id);

CREATE TABLE notification_appeal_links (
  notification_id  BIGINT PRIMARY KEY REFERENCES notifications(notification_id) ON DELETE CASCADE,
  event_type       notification_type NOT NULL, -- appeal_accepted/rejected
  appeal_id        BIGINT NOT NULL REFERENCES appeals(appeal_id) ON DELETE CASCADE
);
CREATE INDEX ix_notification_appeal_links_appeal ON notification_appeal_links(appeal_id);

CREATE TABLE notification_forum_links (
  notification_id   BIGINT PRIMARY KEY REFERENCES notifications(notification_id) ON DELETE CASCADE,
  event_type        notification_type NOT NULL, -- announcement_posted / comment_posted / reply_posted
  forum_post_id     BIGINT REFERENCES forum_posts(post_id) ON DELETE CASCADE,
  forum_comment_id  BIGINT REFERENCES forum_comments(comment_id) ON DELETE CASCADE,
  CONSTRAINT ck_notification_forum_links_one_source CHECK (num_nonnulls(forum_post_id, forum_comment_id) = 1)
);
CREATE INDEX ix_notification_forum_links_post ON notification_forum_links(forum_post_id);
CREATE INDEX ix_notification_forum_links_comment ON notification_forum_links(forum_comment_id);

CREATE TABLE notification_submission_review_links (
  notification_id      BIGINT PRIMARY KEY REFERENCES notifications(notification_id) ON DELETE CASCADE,
  event_type            notification_type NOT NULL, -- submission_assigned/judge_feedback_given
  submission_review_id  BIGINT NOT NULL REFERENCES submission_reviews(review_id) ON DELETE CASCADE
);
CREATE INDEX ix_notification_submission_review_links_review ON
notification_submission_review_links(submission_review_id);
-- NOTE: "notify the whole event" (e.g. an announcement) is a fan-out at
-- insert time -- one notifications row per recipient, each with its
-- own notification_forum_links row pointing at the same forum_post_id
-- -- not a schema concern; nothing here assumes a single recipient.

-- ============================================================================
-- Triggers
-- ============================================================================

-- --- keep updated_at columns honest -------------------------------------
CREATE OR REPLACE FUNCTION set_updated_at() RETURNS trigger AS $$
BEGIN
  NEW.updated_at = now();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_accounts_updated_at BEFORE UPDATE ON accounts
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();
CREATE TRIGGER trg_submissions_updated_at BEFORE UPDATE ON submissions
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- --- subtype/role consistency -------------------------------------------
-- Postgres CHECK constraints can't reference another table, so "a
-- `members` row must belong to an account with role='member'" (and
-- likewise for clubs/judges/partners) is enforced here instead of
-- declaratively. One parameterized function, reused by all four
```

```
-- subtype tables via TG_ARGV, rather than four near-duplicate
-- functions.
CREATE OR REPLACE FUNCTION enforce_subtype_role() RETURNS trigger AS $$
DECLARE
  expected_role account_role := TG_ARGV[0]::account_role;
  actual_role account_role;
BEGIN
  SELECT role INTO actual_role FROM accounts WHERE account_id = NEW.account_id;
  IF actual_role IS DISTINCT FROM expected_role THEN
    RAISE EXCEPTION 'account % has role %, expected % for table %',
      NEW.account_id, actual_role, expected_role, TG_TABLE_NAME;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_members_role_check BEFORE INSERT OR UPDATE ON members
  FOR EACH ROW EXECUTE FUNCTION enforce_subtype_role('member');
CREATE TRIGGER trg_clubs_role_check BEFORE INSERT OR UPDATE ON clubs
  FOR EACH ROW EXECUTE FUNCTION enforce_subtype_role('club');
CREATE TRIGGER trg_judges_role_check BEFORE INSERT OR UPDATE ON judges
  FOR EACH ROW EXECUTE FUNCTION enforce_subtype_role('judge');
CREATE TRIGGER trg_partners_role_check BEFORE INSERT OR UPDATE ON partners
  FOR EACH ROW EXECUTE FUNCTION enforce_subtype_role('partner');

-- --- team capacity by category ----------------------------------------
-- "Robotics = max 5, Programming = max 3" is a count-across-rows rule,
-- so it's enforced on INSERT into memberships rather than as a CHECK.
CREATE OR REPLACE FUNCTION enforce_team_capacity() RETURNS trigger AS $$
DECLARE
  v_max SMALLINT;
  v_current INT;
BEGIN
  SELECT tc.max_members INTO v_max
  FROM teams t JOIN team_categories tc ON tc.category_id = t.category_id
  WHERE t.team_id = NEW.team_id;

  SELECT count(*) INTO v_current
  FROM memberships
  WHERE team_id = NEW.team_id AND left_at IS NULL;

  IF v_current >= v_max THEN
    RAISE EXCEPTION 'team % is at capacity (% members, category max %)', NEW.team_id, v_current, v_max;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_memberships_capacity BEFORE INSERT ON memberships
  FOR EACH ROW EXECUTE FUNCTION enforce_team_capacity();

-- --- max 3 invalid (rejected) appeals per team ---------------------------
CREATE OR REPLACE FUNCTION enforce_appeal_limit() RETURNS trigger AS $$
DECLARE
  v_rejected_count INT;
BEGIN
```

```
  SELECT count(*) INTO v_rejected_count
  FROM appeals
  WHERE team_id = NEW.team_id AND status = 'rejected';

  IF v_rejected_count >= 3 THEN
    RAISE EXCEPTION 'team % has reached the limit of 3 invalid (rejected) appeals', NEW.team_id;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_appeals_limit BEFORE INSERT ON appeals
  FOR EACH ROW EXECUTE FUNCTION enforce_appeal_limit();

-- --- submission resubmission versioning ---------------------------------
-- Assigns version server-side instead of trusting the client with it,
-- so two racing "resubmit" requests for the same team/challenge can't
-- both compute the same next version and collide on
-- uq_submissions_team_challenge_version.
CREATE OR REPLACE FUNCTION assign_submission_version() RETURNS trigger AS $$
BEGIN
  SELECT COALESCE(MAX(version), 0) + 1 INTO NEW.version
  FROM submissions
  WHERE team_id = NEW.team_id AND challenge_id = NEW.challenge_id;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_submissions_version BEFORE INSERT ON submissions
  FOR EACH ROW EXECUTE FUNCTION assign_submission_version();

-- --- exactly one link per notification ----------------------------------
-- The four notification_*_links tables are all 1:1 extensions of
-- notifications (same PK/FK shape as members/clubs/judges/partners
-- extending accounts), but nothing about that shape stops the same
-- notification_id from also getting a row in a second links table by
-- mistake. This checks across all four before allowing an insert into
-- any one of them.
CREATE OR REPLACE FUNCTION enforce_single_notification_link() RETURNS trigger AS $$
DECLARE
  existing_count INT;
BEGIN
  SELECT
    (SELECT count(*) FROM notification_join_request_links WHERE notification_id = NEW.notification_id) +
    (SELECT count(*) FROM notification_appeal_links WHERE notification_id = NEW.notification_id) +
    (SELECT count(*) FROM notification_forum_links WHERE notification_id = NEW.notification_id) +
    (SELECT count(*) FROM notification_submission_review_links WHERE notification_id = NEW.notification_id)
  INTO existing_count;

  IF existing_count > 0 THEN
    RAISE EXCEPTION 'notification % already has a link', NEW.notification_id;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trg_notification_join_request_links_single BEFORE INSERT ON notification_join_request_links
  FOR EACH ROW EXECUTE FUNCTION enforce_single_notification_link();
CREATE TRIGGER trg_notification_appeal_links_single BEFORE INSERT ON notification_appeal_links
  FOR EACH ROW EXECUTE FUNCTION enforce_single_notification_link();
CREATE TRIGGER trg_notification_forum_links_single BEFORE INSERT ON notification_forum_links
  FOR EACH ROW EXECUTE FUNCTION enforce_single_notification_link();
CREATE TRIGGER trg_notification_submission_review_links_single BEFORE INSERT ON
notification_submission_review_links
  FOR EACH ROW EXECUTE FUNCTION enforce_single_notification_link();

-- ======================================================================
-- Optional: Row Level Security example (submissions edit permissions)
-- ======================================================================
-- "A member can edit only their own team's submission; a club can edit
-- any of its teams' submissions" is an authorization rule, not a data
-- integrity rule -- normally enforced in the Node.js service layer.
-- Shown here as an OPTIONAL illustration of expressing it at the DB
-- layer too (defense in depth), using session variables the app sets
-- per request after authenticating (`SET LOCAL app.current_account_id`,
-- `SET LOCAL app.current_role`). Not required for the schema to work;
-- comment out or delete if the API will own this check exclusively.

-- ALTER TABLE submissions ENABLE ROW LEVEL SECURITY;
--
-- CREATE POLICY submissions_team_member_edit ON submissions
--   FOR UPDATE
--   USING (
--     current_setting('app.current_role', true) = 'member'
--     AND team_id IN (
--       SELECT m.team_id FROM memberships m
--       WHERE m.member_id = current_setting('app.current_account_id', true)::BIGINT
--         AND m.left_at IS NULL
--     )
--   );
--
-- CREATE POLICY submissions_club_edit ON submissions
--   FOR UPDATE
--   USING (
--     current_setting('app.current_role', true) = 'club'
--     AND team_id IN (
--       SELECT t.team_id FROM teams t
--       WHERE t.club_id = current_setting('app.current_account_id', true)::BIGINT
--     )
--   );
```